

Customer App Documentation

This document provides a comprehensive overview of the **Nearby Customer App**, its architecture, features, and how it integrates with the Django backend.

App Architecture

The Customer App is built using modern React Native and Expo tooling:

- **Framework:** React Native + Expo
- **Routing:** Expo Router (File-based routing in `src/app/`)
- **Styling:** TailwindCSS via the `twrnc` library (using `tw` `` template tags)
- **Icons:** `lucide-react-native`
- **State Management:**
 - `useSyncExternalStore` for simple local state (e.g., Booking Draft in `lib/bookingState.ts`)
 - Custom `useFetch` hook (`hooks/useFetch.ts`) for data fetching.

API Integration

All network requests are handled through `src/lib/api.ts`, which automatically attaches the user's JWT Token (`Bearer <token>`) using `expo-secure-store`.

Authentication

- **Login:** POST `/api/auth/login/` - Returns access & refresh tokens.
- **Register:** POST `/api/auth/register/`

Core Endpoints (GET)

- **Stores/Salons:** `/api/customer/stores/`
- **Store Details & Menu:** `/api/customer/stores/{id}/menu/`
- **Profile:** `/api/customer/profile/`
- **Appointments:** `/api/customer/appointments/`
- **Wallet & Transactions:** `/api/customer/wallet/`
- **Offers:** `/api/customer/offers/`
- **Packages:** `/api/customer/packages/`

- **Memberships:** `/api/customer/memberships/`
- **Gift Cards:** `/api/customer/giftcards/`
- **Recommended Therapists:** `/api/customer/recommended-therapists/`

Creating an Appointment (POST)

To book a new appointment, a `POST` request is sent to `/api/customer/appointments/` with the following JSON structure:

```
{
  "store": 1,
  "date": "2026-07-24",
  "start_time": "14:00:00",
  "notes": "Booked via CustomerApp (salon)",
  "items": [
    { "service": 5 }
  ]
}
```



Navigation & File Structure

The app uses Expo's file-based routing mechanism inside the `src/app/` directory:

```
src/app/
├── _layout.tsx           # Root layout (handles auth guarding)
├── login.tsx             # Authentication screen
├── (tabs)/               # Main Bottom Tab Navigation
│   ├── index.tsx         # Dashboard / Home Screen
│   ├── explore.tsx       # Search & Discovery Map
│   ├── bookings.tsx      # Upcoming and Past Appointments
│   └── profile.tsx       # User Profile & Settings
├── salon/
│   └── [id].tsx           # Salon Details & Service Menu
├── book/
│   └── [salonId]/         # Multi-step Booking Wizard
│       ├── service.tsx   # Step 1: Pick a service
│       ├── therapist.tsx # Step 2: Pick a therapist
│       ├── time.tsx      # Step 3: Pick a date & time
│       └── pay.tsx        # Step 4: Confirm & POST to API
├── booking/
│   └── [id]/index.tsx     # Appointment Details
```

```
|   └─ [id]/reschedule.tsx      # Reschedule Flow (Coming Soon)
└─ profile/                    # Profile Sub-screens
    └─ personal.tsx            # Personal Information
    └─ payments.tsx            # Saved Payment Methods
    └─ ...                      # Wallet, Offers, Memberships, Packages, etc.
```

🚧 Graceful Fallbacks (Empty States)

To ensure the app never crashes, several screens that do not currently have Django backend endpoints have been built with clean "Empty State" UIs. These features are ready to be integrated when the backend APIs are built in the future:

- **Chat / Messages** (`/app/messages/`)
- **Reviews** (`/app/reviews.tsx`)
- **Favourites** (`/app/favourites.tsx`)
- **Rewards** (`/app/rewards.tsx`)
- **Notifications** (`/app/notifications.tsx`)
- **AI Stylist** (`/app/ai-beauty/`)